



PROJET ACADÉMIQUE DE MASTER

CARTOGRAPHIE SÉMANTIQUE ET ANALYSE DES PUBLICATIONS DE LA FSBM PAR NLP & WEB SCRAPING

*Conception d'un pipeline de collecte, de traitement et de recherche
sémantique des publications scientifiques*

Réalisé par

ALAA ZOUBIR

Université Hassan II de Casablanca
Faculté des Sciences Ben M'Sick — FSBM

Année universitaire 2025-2026

Résumé

La croissance continue de la production scientifique rend difficile l'exploration des publications d'un établissement, d'autant que les outils de recherche classiques reposent sur une correspondance lexicale entre les mots de la requête et ceux des documents. Ce projet s'intéresse aux publications associées à des profils de chercheurs de la Faculté des Sciences Ben M'Sick (FSBM), Université Hassan II de Casablanca, et vise à les rendre explorables par le sens.

Un pipeline complet a été conçu à cet effet. Les profils et publications sont collectés depuis Google Scholar avec la bibliothèque Python `scholarly`, selon une stratégie prudente fondée sur des délais, des reprises avec backoff exponentiel, des checkpoints atomiques et un arrêt en cas de blocage, sans aucun contournement des restrictions de la source. Les données sont ensuite consolidées, normalisées et dédoublées avec `pandas`, puis enregistrées aux formats JSON et Parquet accompagnées d'un rapport de qualité. Le corpus obtenu compte 77 profils, 1 044 enregistrements bruts, 959 publications uniques après fusion de 85 doublons, et 895 publications disposant d'un résumé exploitable.

Ces 895 résumés nettoyés sont encodés avec le modèle multilingue `zeroentropy/zembed-1-embedding` en vecteurs de 2 560 dimensions, puis indexés dans la base vectorielle ChromaDB avec une distance cosinus. Une recherche sémantique Top-K encode la requête de l'utilisateur et renvoie les publications les plus proches. L'ensemble est exposé par une API FastAPI et une interface web React, TypeScript et Vite comprenant un tableau de bord, la consultation des chercheurs et des publications et la recherche sémantique. Le projet est validé par 46 tests Python et 5 tests frontend.

Les résultats montrent une recherche fonctionnelle de bout en bout, dont la pertinence varie toutefois selon la couverture du corpus et la qualité des résumés. Les limites de la collecte, notamment son caractère partiel et l'absence de vérification de l'appartenance institutionnelle, sont documentées explicitement.

Mots-clés : Web Scraping, NLP, Google Scholar, Embeddings, zembed-1, Recherche sémantique, ChromaDB, FastAPI, React

Abstract

The steady growth of scientific output makes it difficult to explore the publications of an institution, especially since conventional search tools rely on lexical matching between query terms and document terms. This project focuses on publications associated with researcher profiles of the Faculté des Sciences Ben M'Sick (FSBM), Hassan II University of Casablanca, and aims to make them explorable by meaning.

A complete pipeline was designed for this purpose. Researcher profiles and publications are collected from Google Scholar with the Python library `scholarly`, using a conservative strategy based on delays, retries with exponential backoff, atomic checkpoints and a stop on blocking, without any circumvention of the source's restrictions. The data are then consolidated, normalized and deduplicated with pandas, and stored as JSON and Parquet files together with a data quality report. The resulting corpus contains 77 profiles, 1,044 raw records, 959 unique publications after merging 85 duplicates, and 895 publications with a usable abstract.

These 895 cleaned abstracts are encoded with the multilingual model `zeroentropy/zembed-1-embedding` into 2,560-dimensional vectors, which are indexed in the ChromaDB vector database using cosine distance. A Top-K semantic search encodes the user query and returns the closest publications. The system is exposed through a FastAPI service and a React, TypeScript and Vite web interface providing a dashboard, researcher and publication browsing, and semantic search. The project is validated by 46 Python tests and 5 frontend tests.

The results show a working end-to-end search whose relevance nevertheless depends on corpus coverage and abstract quality. The limitations of the collection, in particular its partial nature and the lack of verification of institutional affiliation, are explicitly documented.

Keywords: Web Scraping, NLP, Google Scholar, Embeddings, zembed-1, Semantic Search, ChromaDB, FastAPI, React

Table des matières

Introduction générale.....	1
1. Contexte et objectifs du projet.....	2
1.1 Contexte général.....	2
1.2 Problématique.....	2
1.3 Objectifs.....	2
1.4 Architecture générale.....	3
2. Collecte des données.....	5
2.1 Source de données : Google Scholar.....	5
2.2 Informations collectées.....	5
2.3 Stratégie de web scraping.....	6
2.4 Robustesse du scraper.....	6
2.5 Résultat de la collecte.....	8
3. Prétraitement et qualité des données.....	9
3.1 Données brutes.....	9
3.2 Nettoyage.....	9
3.3 Normalisation.....	9
3.4 Déduplication.....	9
3.5 Consolidation.....	10
3.6 Formats de sortie JSON et Parquet.....	11
3.7 Contrôle qualité.....	11
4. Génération des embeddings.....	13
4.1 Principe des embeddings.....	13
4.2 Intérêt pour la recherche scientifique.....	13
4.3 Modèle zembed-1.....	13
4.4 Préparation du texte.....	13
4.5 Encodage des documents.....	14
4.6 Dimension des vecteurs.....	14
4.7 Génération finale.....	14
5. Indexation et recherche sémantique.....	16
5.1 Principe de la recherche sémantique.....	16
5.2 ChromaDB.....	16
5.3 Index vectoriel.....	16
5.4 Similarité cosinus.....	17
5.5 Encodage d'une requête.....	17
5.6 Recherche Top-K.....	17
6. Conception et réalisation de l'application.....	19

6.1 Backend FastAPI	19
6.2 Frontend	20
6.3 Tableau de bord	20
6.4 Consultation des chercheurs	21
6.5 Consultation des publications.....	22
6.6 Recherche sémantique	22
7. Tests, validation et résultats.....	24
7.1 Tests automatisés.....	24
7.2 Validation des embeddings et de l'index	25
7.3 Analyse des résultats de la recherche sémantique	25
8. Enrichissement PDF et limitations	26
8.1 Module d'enrichissement PDF optionnel.....	26
8.2 Résultats de l'enrichissement	26
8.3 Limitations générales	27
Conclusion générale et perspectives.....	28
Références bibliographiques	29

Liste des figures

Figure 1 - Architecture globale du pipeline de collecte, prétraitement, indexation et recherche sémantique.....	3
Figure 2 - Processus de collecte robuste des données depuis Google Scholar	7
Figure 3 - Prétraitement, déduplication et préparation du corpus pour l'indexation sémantique	10
Figure 4 - Répartition des publications uniques par année de publication	12
Figure 5 - Processus de génération des embeddings avec zembed-1.....	15
Figure 6 - Processus de recherche sémantique et classement des publications par similarité.....	17
Figure 7 - Architecture logicielle de l'application de démonstration	19
Figure 8 - Tableau de bord de l'application FSBM Semantic Research	21
Figure 9 - Consultation des chercheurs et de leurs indicateurs scientifiques.....	21
Figure 10 - Consultation et exploration des publications scientifiques.....	22
Figure 11 - Exemple de recherche sémantique et classement des publications selon leur similarité	23

Liste des tableaux

Tableau 1 - Technologies utilisées dans le projet	4
Tableau 2 - Informations collectées pour les profils et les publications	5
Tableau 3 - Répartition des données brutes par fichier source	8
Tableau 4 - Synthèse quantitative du corpus.....	10
Tableau 5 - Complétude des métadonnées des publications uniques	11
Tableau 6 - Caractéristiques de l'artefact d'embeddings final.....	15
Tableau 7 - Configuration de l'index vectoriel ChromaDB.....	16
Tableau 8 - Endpoints de l'API REST	19
Tableau 9 - Pages de l'interface web	20
Tableau 10 - Résultats des validations automatisées.....	24
Tableau 11 - Répartition des tests Python par module	24
Tableau 12 - Statuts de l'exécution d'enrichissement PDF (231 publications vérifiées)	26
Tableau 13 - Limitations rencontrées et mesures mises en place	27

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DOI	Digital Object Identifier
FSBM	Faculté des Sciences Ben M'Sick
HNSW	Hierarchical Navigable Small World
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
NDCG	Normalized Discounted Cumulative Gain
NFKC	Normalization Form Compatibility Composition (Unicode)
NLP	Natural Language Processing
NPZ	Format d'archive compressée de tableaux NumPy
PDF	Portable Document Format
REST	Representational State Transfer
SHA	Secure Hash Algorithm
URL	Uniform Resource Locator

Introduction générale

La production scientifique mondiale connaît depuis plusieurs décennies une croissance soutenue. Les analyses bibliométriques fondées sur le nombre de publications et de références citées montrent que le volume de la littérature scientifique augmente de manière continue, avec un taux de croissance annuel de l'ordre de quelques pourcents [1]. Les établissements universitaires marocains n'échappent pas à cette dynamique, avec des travaux dispersés entre revues, conférences et plateformes bibliographiques.

Explorer cet ensemble reste pourtant difficile. Les outils de recherche classiques reposent majoritairement sur une correspondance lexicale entre les mots de la requête et ceux des documents [2], [3]. Cette approche est efficace lorsque l'utilisateur connaît le vocabulaire exact employé par les auteurs, mais elle montre ses limites dès qu'un même sujet est décrit avec des termes différents, dans des langues différentes ou à des niveaux d'abstraction variés. Une requête sur « l'analyse automatique du texte » ne retrouvera pas nécessairement un article qui parle de « traitement du langage naturel », alors que les deux relèvent du même domaine.

Le traitement automatique du langage naturel (NLP) propose une réponse à ce problème à travers les représentations vectorielles denses, appelées *embeddings*. Un modèle d'embeddings associe à chaque texte un vecteur numérique de grande dimension, construit de telle sorte que deux textes proches par leur sens soient représentés par des vecteurs proches dans l'espace [4]–[6]. Il devient alors possible de comparer une requête et des documents par leur signification plutôt que par leurs mots, et donc de construire une *recherche sémantique* ou une *cartographie sémantique* d'un corpus [7].

Ce projet applique cette démarche aux publications associées à des profils de chercheurs de la Faculté des Sciences Ben M'Sick (FSBM), Université Hassan II de Casablanca. Le caractère pluridisciplinaire de la faculté rend cette navigation particulièrement intéressante : un même thème transversal, comme l'environnement ou l'intelligence artificielle, peut apparaître dans des publications de départements différents.

La problématique retenue peut se formuler ainsi : **comment constituer, à partir de sources bibliographiques publiques, un corpus structuré et fiable de publications liées à la FSBM, puis le rendre explorable par le sens grâce aux techniques de NLP, tout en respectant les contraintes d'accès aux données et en documentant honnêtement les limites du corpus obtenu ?**

Pour y répondre, le projet met en place un pipeline complet : collecte de profils et de publications depuis Google Scholar à l'aide de la bibliothèque `scholarly`, consolidation et nettoyage des données avec `pandas`, déduplication, génération d'embeddings avec le modèle `zeroentropy/zembed-1-embedding`, indexation dans la base vectorielle ChromaDB, recherche par similarité cosinus, puis exposition des résultats à travers une API FastAPI et une interface web React.

Le rapport est organisé en huit chapitres. Le chapitre 1 présente le contexte, les objectifs et l'architecture générale du projet. Le chapitre 2 décrit la collecte des données depuis Google Scholar et les mécanismes de robustesse du scraper. Le chapitre 3 est consacré au

prétraitement, à la déduplication et au contrôle qualité du corpus. Le chapitre 4 explique la génération des embeddings, et le chapitre 5 l'indexation vectorielle et la recherche sémantique. Le chapitre 6 présente l'application de démonstration, le chapitre 7 les tests et l'analyse des résultats, et le chapitre 8 l'enrichissement PDF optionnel et les limitations, avant une conclusion ouverte sur des perspectives.

1. Contexte et objectifs du projet

Ce premier chapitre situe le projet dans son contexte, formule précisément la problématique et les objectifs poursuivis, puis présente l'architecture générale du pipeline mis en œuvre. Il sert de fil conducteur aux chapitres suivants, qui détaillent chacune des étapes.

1.1 Contexte général

Les indicateurs bibliométriques disponibles publiquement, comme le nombre de citations ou l'indice h [9], renseignent sur l'activité d'un chercheur mais disent peu de choses sur le contenu de ses travaux. Or, pour identifier des collaborations possibles, orienter un étudiant vers un encadrant ou simplement comprendre les thématiques présentes dans une faculté, c'est bien le contenu scientifique qui importe. Les résumés des publications constituent à cet égard une source d'information précieuse : ils condensent en quelques phrases l'objet, la méthode et les résultats d'un travail.

Google Scholar est l'une des sources les plus accessibles pour recenser les publications d'un chercheur, car de nombreux enseignants-chercheurs y maintiennent un profil public. Il ne propose toutefois pas d'API officielle, applique des limitations d'accès strictes et ne fournit pas de vue consolidée à l'échelle d'un établissement. Construire une vue sémantique des publications de la FSBM suppose donc d'assembler plusieurs briques : une collecte prudente, un nettoyage rigoureux, une représentation sémantique des textes et un outil d'exploration.

1.2 Problématique

La difficulté ne se limite pas à la recherche d'information. Elle porte d'abord sur la constitution du corpus lui-même. Les données publiques sont incomplètes (résumés absents ou tronqués, métadonnées manquantes), redondantes (une même publication peut apparaître sur les profils de plusieurs co-auteurs) et parfois ambiguës (une affiliation déclarée sur Google Scholar n'établit pas à elle seule l'appartenance actuelle d'un chercheur à la FSBM). À cela s'ajoutent les contraintes techniques et éthiques de la collecte : limitation de débit, CAPTCHA et conditions d'utilisation de la source.

La question centrale est donc double : obtenir un corpus exploitable, traçable et correctement dédoublé, puis en permettre une exploration par le sens qui reste transparente sur ce qu'elle mesure réellement, à savoir une similarité entre une requête et des résumés disponibles.

1.3 Objectifs

L'objectif principal du projet est de **construire un pipeline permettant de collecter, nettoyer, structurer, vectoriser et explorer sémantiquement des publications scientifiques associées aux profils étudiés**. Cet objectif se décline en plusieurs objectifs spécifiques :

- collecter des profils de chercheurs sur Google Scholar à partir d'une liste d'identifiants, en conservant les métriques disponibles (citations, indice h , indice $i10$) ;
- collecter les publications de chaque profil avec leurs métadonnées (titre, auteurs, date, revue ou conférence, citations, résumé lorsqu'il est disponible) ;

- nettoyer, normaliser et structurer ces données, en préservant les valeurs originales et leur provenance ;
- détecter et fusionner les doublons afin d'obtenir un jeu de publications uniques ;
- constituer un dataset consolidé aux formats JSON et Parquet, accompagné d'un rapport de qualité ;
- générer des embeddings pour les publications disposant d'un résumé exploitable ;
- indexer ces vecteurs dans une base vectorielle et implémenter une recherche sémantique Top-K par similarité cosinus ;
- exposer le corpus et la recherche à travers une API REST ;
- proposer une interface web de démonstration permettant de consulter les chercheurs, les publications et d'effectuer des recherches sémantiques.

1.4 Architecture générale

L'architecture retenue suit une logique de pipeline : chaque étape consomme les sorties de la précédente et produit des artefacts persistants, ce qui permet de reprendre ou de reproduire une étape sans relancer l'ensemble. La Figure 1 en donne une vue d'ensemble, depuis la source Google Scholar jusqu'à l'application de recherche.



Figure 1 - Architecture globale du pipeline de collecte, prétraitement, indexation et recherche sémantique

La chaîne comporte sept étapes. Les **profils et publications** sont d'abord lus sur Google Scholar par un module de **web scraping** contrôlé, qui gère les erreurs et sauvegarde sa progression. Les données brutes passent ensuite par un **prétraitement** (nettoyage, normalisation, déduplication) qui aboutit à un **dataset final** de 959 publications uniques, dont 895 disposent d'un résumé exploitable. Ces 895 publications sont encodées par le modèle **zembed-1** en vecteurs de 2 560 dimensions, puis stockées et indexées dans **ChromaDB**. Enfin, la **recherche sémantique** encode la requête de l'utilisateur, interroge l'index et renvoie les publications les plus proches, affichées dans l'application FastAPI + React.

Cette figure est une illustration pédagogique. Sur un point, elle simplifie la réalité du code : l'étape `zembed-1` y mentionne l'encodage « des titres + résumés (FR/EN) ». Dans l'implémentation finale, le texte encodé est uniquement le résumé nettoyé (`abstract_clean`), sans concaténation du titre et sans étape de traduction ; le titre reste une métadonnée associée au vecteur. Ce point est détaillé au chapitre 4.

Les technologies mobilisées à chaque étape sont récapitulées dans le Tableau 1. Elles correspondent aux dépendances déclarées dans `requirements.txt` et `frontend/package.json`.

Tableau 1 - Technologies utilisées dans le projet

Étape	Technologie	Rôle dans le projet
Langage principal	Python	Collecte, prétraitement, embeddings, recherche et API
Collecte	scholarly	Accès aux profils et publications Google Scholar
Traitement des données	pandas, NumPy	Consolidation, contrôle des lignes, manipulation des vecteurs
Stockage structuré	JSON, Apache Parquet (pyarrow)	Données brutes, corpus nettoyé et rapports
Embeddings	sentence-transformers, PyTorch	Chargement et exécution de <code>zeroentropy/zembed-1-embedding</code>
Base vectorielle	ChromaDB	Index HNSW persistant, distance cosinus
API	FastAPI, Uvicorn, Pydantic	Service REST en lecture seule (méthodes GET)
Frontend	React, TypeScript, Vite, react-router	Interface web de démonstration
Tests	unittest, Vitest, Testing Library	Tests Python hors ligne et tests frontend simulés
Analyse	Jupyter, Matplotlib	Notebook de démonstration et visualisations

Ce choix d'outils privilégie des composants open source, bien documentés et exécutables localement. Aucun système de gestion de base de données relationnelle n'est utilisé : les données structurées sont conservées sous forme de fichiers JSON et Parquet versionnés, et seule la base vectorielle ChromaDB est employée pour l'indexation sémantique.

2. Collecte des données

La qualité de tout le pipeline dépend de la collecte initiale. Ce chapitre décrit la source de données, les informations effectivement récupérées, la stratégie de scraping adoptée, les mécanismes de robustesse mis en place et, enfin, le résultat obtenu avec ses limites.

2.1 Source de données : Google Scholar

Google Scholar indexe une large part de la littérature scientifique et permet aux chercheurs de créer un profil public listant leurs publications et leurs indicateurs de citation. Chaque profil est identifié par un **Scholar ID**, une chaîne de douze caractères (par exemple `upOdTrEAAAAJ`). Le projet part d'une liste de chercheurs fournie en entrée (`data/input/input_faculty_list.json`), qui contient 97 lignes et 93 identifiants distincts, quatre identifiants y figurant deux fois.

Google Scholar ne dispose pas d'API publique officielle. La collecte s'appuie donc sur la bibliothèque Python `scholarly` [13], qui interroge les pages publiques des profils et en extrait les informations structurées. Ce choix implique de respecter strictement les limitations imposées par la source, point développé plus loin.

2.2 Informations collectées

Pour chaque chercheur et chaque publication, le scraper enregistre les champs disponibles sur la page source ainsi que des informations de provenance. Le Tableau 2 synthétise ces champs. Il est important de souligner que **tous les champs ne sont pas renseignés pour toutes les entrées** : leur présence dépend de ce que Google Scholar expose pour chaque profil et chaque publication.

Tableau 2 - Informations collectées pour les profils et les publications

Niveau	Champs	Remarques
Profil chercheur	Scholar ID, nom, affiliation	L'affiliation est une chaîne déclarée par le chercheur, non vérifiée
Profil chercheur	Centres d'intérêt	Renseignés pour 38 des 77 profils du corpus final
Profil chercheur	Citations totales, indice h, indice i10	Métriques calculées par Google Scholar au moment de la collecte
Publication	Titre, auteurs, date ou année	Année absente pour 17 publications uniques
Publication	Revue ou conférence, volume, pages, éditeur	Revue/conférence absente pour 76 publications uniques
Publication	Nombre de citations	Présent pour toutes les publications uniques
Publication	Résumé (abstract)	Souvent tronqué par la source ; absent pour une partie des notices
Publication	URL de la publication, URL Scholar	Pas fournies par la source historique (legacy)

Niveau	Champs	Remarques
Provenance	Sources des métadonnées et du résumé, horodatage, statut	Permettent de tracer l'origine de chaque information

Ce tableau met en évidence un choix de conception important : le scraper conserve, à côté des données, la **provenance** de chaque information (URL du profil, URL de la notice, fichier brut d'origine) et un **statut** de collecte. Ces éléments ne servent pas directement à la recherche sémantique, mais ils garantissent la traçabilité du corpus et permettent d'expliquer a posteriori pourquoi une information est manquante.

2.3 Stratégie de web scraping

Le module `src/scraping/scrapper.py` traite une liste de chercheurs sélectionnés. Pour chaque identifiant, il récupère le profil par `scholarly.search_author_id`, puis le complète avec la fonction `scholarly.fill` sur les sections `basics`, `indices` et `publications`. Chaque publication listée sur le profil est ensuite complétée individuellement, toujours par `scholarly.fill`, afin d'obtenir le détail de la notice, notamment le résumé lorsqu'il existe.

Les enregistrements sont transformés en un format homogène par la fonction `publication_record`, qui construit un identifiant d'article de la forme `ScholarID:identifiant_de_publication`, reconstruit l'URL de la notice sur Google Scholar et renseigne les champs de provenance. Le scraper ne réécrit jamais le fichier de référence historique `raw_scholar_data.json` : chaque campagne de collecte écrit dans son propre fichier de checkpoint.

La collecte s'est déroulée en plusieurs lots successifs, qui alimentent un **unique dataset consolidé**. Le fichier `raw_scholar_data.json` constitue l'échantillon initial (format historique, dit *legacy*) ; les fichiers `new_scholar_data.json`, `new_scholar_data_batch2.json` et `new_scholar_data_batch3.json` correspondent aux campagnes suivantes, au format checkpoint. Des scripts dédiés (`validate_batch2.py`, `validate_batch3.py`) vérifient, avant chaque lot, que les identifiants sélectionnés ne recouvrent pas ceux déjà collectés et qu'ils sont cohérents avec les noms attendus.

2.4 Robustesse du scraper

Google Scholar limite fortement les accès automatisés. Le scraper a donc été conçu pour être prudent et pour ne jamais perdre le travail déjà effectué. La Figure 2 illustre ce processus.

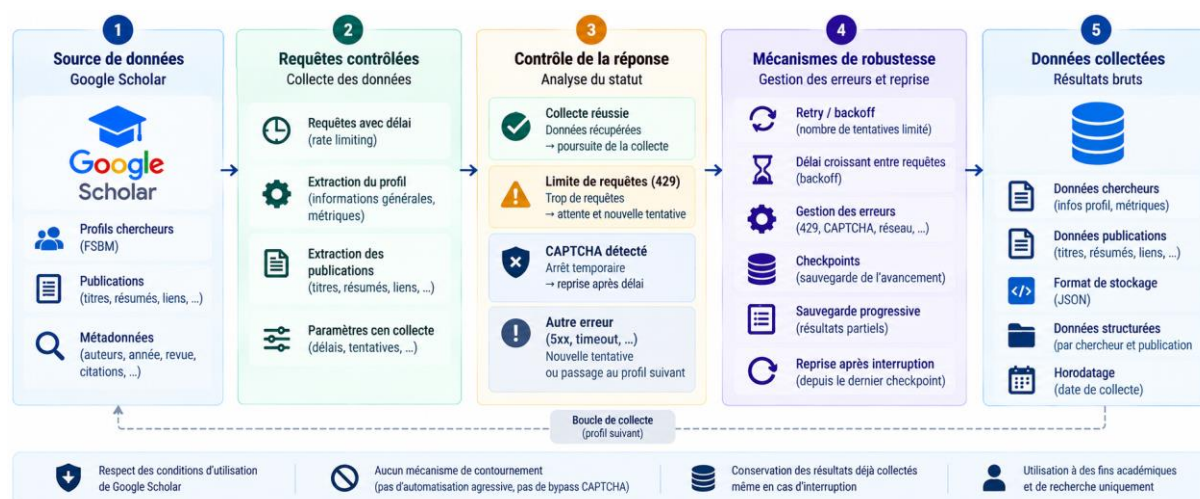


Figure 2 - Processus de collecte robuste des données depuis Google Scholar

Les mécanismes réellement implémentés dans le code sont les suivants :

- **Délais entre requêtes** : un délai minimal configurable (5 secondes par défaut, option `--delay`) est appliqué entre les appels, augmenté d'une petite composante aléatoire.
- **Retries avec backoff exponentiel** : en cas d'erreur transitoire, la fonction `retry` relance l'appel un nombre limité de fois (3 par défaut), avec une attente qui double à chaque tentative.
- **Détection des blocages** : les erreurs évoquant un CAPTCHA, un code HTTP 429 (*Too Many Requests*) ou 403, ou un refus d'accès sont converties en exception `ScholarBlocked`. Dans ce cas, le scraper ne réessaie pas indéfiniment : il enregistre l'échec et **arrête la campagne en cours**.
- **Checkpoints atomiques** : l'état de la collecte est sauvegardé après chaque publication dans un fichier temporaire, puis substitué au checkpoint par `os.replace`. Une interruption ne peut donc pas corrompre le fichier existant.
- **Conservation des résultats partiels** : si un blocage survient au milieu d'un profil, les publications déjà complétées sont gardées et les suivantes sont conservées sous forme de notices minimales (statut `not_filled_blocked`).
- **Reprise** : au lancement suivant, les profils marqués `complete` sont ignorés ; l'option `--skip-blocked` permet d'écarter les identifiants précédemment bloqués.
- **Limitation volontaire** : l'option `--max-publications` permet de ne compléter qu'une partie des publications d'un profil, qui reçoit alors le statut `partial_limited`.

La figure présente le cas HTTP 429 comme une attente suivie d'une nouvelle tentative. Dans le code, ce comportement correspond aux erreurs transitoires ordinaires ; lorsqu'un signal explicite de blocage (429, CAPTCHA, 403) est détecté, la politique retenue est plus conservatrice : la campagne s'arrête et reprendra plus tard à partir du dernier checkpoint. **Aucun mécanisme de contournement n'a été implémenté** : pas de rotation de proxys, pas de résolution de CAPTCHA, pas d'authentification détournée. Ce choix réduit le volume collecté, mais il respecte les conditions d'accès de la source et l'usage strictement académique du projet.

2.5 Résultat de la collecte

Au terme des différentes campagnes, 77 profils distincts ont été collectés, soit 77 des 93 identifiants distincts de la liste d'entrée. Ces profils totalisent 1 044 enregistrements bruts de publications, répartis entre les quatre fichiers sources comme le montre le Tableau 3.

Tableau 3 - Répartition des données brutes par fichier source

Fichier source	Format	Profils	Publications
raw_scholar_data.json	legacy	37	430
new_scholar_data.json	checkpoint	9	176
new_scholar_data_batch2.json	checkpoint	10	131
new_scholar_data_batch3.json	checkpoint	21	307
Total		77	1 044

Les statuts enregistrés dans les checkpoints renseignent sur la complétude de la collecte : 37 profils proviennent de l'échantillon historique, 8 ont été collectés complètement, 31 ont le statut `partial_limited` (collecte volontairement limitée à une partie des publications) et 1 est resté `in_progress`. Un identifiant de la première campagne a par ailleurs été bloqué par Google Scholar et n'a fourni aucune publication. Au total, 76 des 77 profils sont représentés par au moins une publication dans le corpus final.

Plusieurs limites doivent être gardées à l'esprit. D'abord, **la collecte est partielle** : pour de nombreux profils, seule une partie des publications a été complétée, et 16 identifiants de la liste d'entrée ne figurent pas dans le corpus. Ensuite, les données disponibles sont inégales : le format historique ne contient ni URL de publication, ni références bibliographiques. Enfin, **l'appartenance institutionnelle n'est pas vérifiée** : l'affiliation provient de la déclaration du chercheur sur Google Scholar. Un simple repérage des mentions « FSBM » ou « Ben M'Sick » dans ces chaînes montre que 28 profils sur 77 citent explicitement la faculté ; les autres mentionnent l'Université Hassan II sans préciser l'établissement, une autre institution, ou aucune information exploitable. Le corpus doit donc être compris comme un **échantillon de publications associées aux profils étudiés**, et non comme la bibliographie exhaustive et certifiée de la FSBM.

3. Prétraitement et qualité des données

Les données brutes issues des quatre fichiers sources ne sont pas directement exploitables : elles utilisent deux schémas différents, contiennent des doublons et présentent des champs manquants ou hétérogènes. Ce chapitre décrit les traitements appliqués pour obtenir un corpus propre, puis les contrôles de qualité réalisés.

3.1 Données brutes

Deux schémas coexistent. L'échantillon historique est une liste de profils dont les champs sont nommés en français (`chercheur_id`, `nom_complet`, `articles`, `metriques`, `titre`, `auteurs`). Les fichiers de checkpoint sont des objets contenant une liste `profiles` et un dictionnaire `failures`, avec des champs nommés en anglais (`scholar_id`, `full_name`, `publications`). Le module `src/preprocessing/merge_raw_data.py`, qui s'appuie sur `pandas` [14], lit ces quatre sources sans les modifier, reconnaît automatiquement leur schéma et fusionne les profils par Scholar ID.

3.2 Nettoyage

Le nettoyage est implémenté dans `src/preprocessing/cleaner.py`. Il suit un principe de prudence : **les valeurs originales sont toujours conservées**, et les versions nettoyées sont stockées dans des champs distincts. Le champ `abstract` garde ainsi le résumé exactement tel qu'il a été collecté, tandis que `abstract_clean` en contient la version normalisée.

La fonction `normalize_text` applique successivement le décodage des entités HTML, la suppression des balises, la normalisation Unicode NFKC, le remplacement des caractères de contrôle et du caractère de remplacement Unicode par des espaces, la réduction des espaces multiples et la conversion en minuscules. Les accents ne sont pas supprimés, ce qui préserve le sens des textes rédigés en français. Les valeurs vides ou conventionnelles (« unknown », « n/a », « null », « none ») sont traitées comme manquantes.

3.3 Normalisation

Chaque publication est transformée en une ligne plate d'un schéma commun comportant 34 colonnes, qui réunit les informations du chercheur et celles de la publication. Les auteurs sont convertis en liste, l'année de publication est extraite de la date par une expression régulière (années comprises entre 1900 et 2099), et un titre normalisé (`title_normalized`) est calculé en supprimant ponctuation et variations de casse. Ce titre normalisé ne remplace pas le titre original : il sert uniquement à comparer les notices lors de la déduplication.

3.4 Déduplication

Une même publication peut apparaître plusieurs fois : sur les profils de plusieurs co-auteurs, ou deux fois dans un même profil. La déduplication a été conçue pour éviter les fusions abusives. Deux notices sont considérées comme identiques si l'une des conditions suivantes est remplie :

- elles partagent le même **DOI**, extrait de l'URL ou de l'identifiant ;
- elles ont le même identifiant d'article pour le même chercheur ;

- à défaut de DOI divergent, elles ont un **titre normalisé strictement identique** (d'au moins 12 caractères et trois mots), **la même année** de publication, **et au moins un auteur commun**.

Il ne s'agit donc pas d'une similarité approximative entre titres : la comparaison exige une égalité exacte après normalisation, complétée par une preuve d'auteur. Les cas incertains restent séparés. Avant la comparaison, les notices sont triées par qualité (présence et longueur du résumé, nombre de métadonnées renseignées), de sorte que la version conservée est la plus complète. La fonction `merge_duplicate` complète ensuite cette version avec les informations manquantes de la notice fusionnée et conserve les liens vers tous les chercheurs et identifiants concernés (`researcher_ids`, `article_ids`, `raw_sources`). Cette étape a permis de fusionner **85 doublons**.

3.5 Consolidation

À l'issue de la déduplication, 959 publications uniques sont obtenues. Une publication est ensuite déclarée **éligible aux embeddings** si elle possède un titre et un résumé nettoyé d'au moins 20 caractères. Dans le corpus final, 895 publications satisfont ce critère ; les 64 autres n'ont pas de résumé exploitable (motif `missing_abstract`). Ces 64 publications ne sont pas supprimées : elles restent dans le dataset consolidé, consultables dans l'application, et sont documentées dans `publications_excluded.json`. Elles sont seulement écartées de la vectorisation, puisqu'un embedding calculé sur un texte vide n'aurait aucun sens. La Figure 3 résume ce parcours.

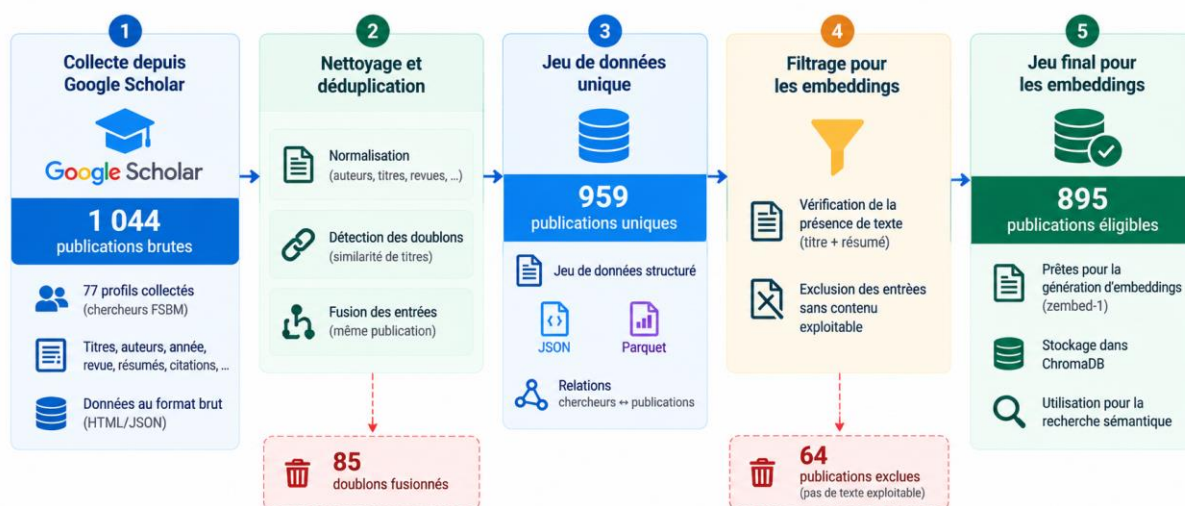


Figure 3 - Prétraitement, déduplication et préparation du corpus pour l'indexation sémantique

La figure met en évidence les deux points de réduction du corpus : la fusion des doublons (1 044 → 959) et le filtrage pour les embeddings (959 → 895). La mention « similarité de titres » qui y figure doit être comprise au sens strict décrit plus haut, c'est-à-dire une égalité de titres normalisés confirmée par l'année et les auteurs. Le Tableau 4 récapitule les chiffres clés, tels qu'ils figurent dans `data_quality_report.json`.

Tableau 4 - Synthèse quantitative du corpus

Indicateur	Valeur
Profils de chercheurs collectés	77
Profils représentés dans les publications	76
Enregistrements bruts de publications	1 044
Doublons fusionnés	85
Publications uniques	959
Publications éligibles aux embeddings	895
Publications exclues des embeddings (résumé absent)	64
Publications uniques liées à plusieurs chercheurs	73

La cohérence arithmétique de ces chiffres est vérifiée par le code lui-même : le script de consolidation lève une erreur si le nombre de lignes brutes ne correspond pas à la somme des publications uniques et des doublons ($959 + 85 = 1\,044$), et l'on a bien $895 + 64 = 959$ publications uniques.

3.6 Formats de sortie JSON et Parquet

Le corpus consolidé est écrit dans le dossier `data/clean/` sous plusieurs formes. Le fichier `researchers.json` contient les 77 profils ; `publications.json` et `publications.parquet` contiennent les 959 publications uniques avec leur indicateur d'éligibilité ; `publications_excluded.json` liste les 149 enregistrements écartés (85 doublons et 64 publications sans résumé) avec leur motif ; enfin `data_quality_report.json` et sa version Markdown rassemblent les indicateurs de qualité.

Le format JSON est lisible et directement consommé par l'API. Le format Parquet [15], colonnaire et typé, est utilisé par l'étape d'embeddings et par le notebook d'analyse. Après écriture, le script relit le fichier Parquet et vérifie le nombre de lignes, la présence de toutes les colonnes et la conservation exacte des titres et résumés (aller-retour Unicode).

3.7 Contrôle qualité

Le rapport de qualité mesure la complétude des principales métadonnées sur les 959 publications uniques. Le Tableau 5 en présente les résultats.

Tableau 5 — Complétude des métadonnées des publications uniques

Champ	Disponible	Manquant
Auteurs	959	0
Nombre de citations	959	0
Année de publication	942	17
Revue ou conférence	883	76

Champ	Disponible	Manquant
URL de la publication	547	412
Résumé exploitable	895	64
URL de PDF validée	0	959
Références bibliographiques	0	959

Les auteurs et les citations sont toujours présents, et l'année ne manque que pour 17 publications. En revanche, aucune URL de PDF validée ni aucune liste de références n'est disponible dans le dataset principal : ces informations ne sont pas fournies de façon systématique par la source. Le rapport ne signale ni titre manquant, ni caractère de remplacement dans les résumés, ni année aberrante. Une observation complémentaire mérite d'être mentionnée : 490 des 895 résumés éligibles se terminent par des points de suspension, ce qui indique qu'ils ont été tronqués par la source. La longueur des résumés nettoyés varie de 69 à 1 699 caractères, avec une médiane de 915 caractères.

La répartition temporelle des publications uniques, présentée dans la Figure 4, donne un aperçu de la composition du corpus.

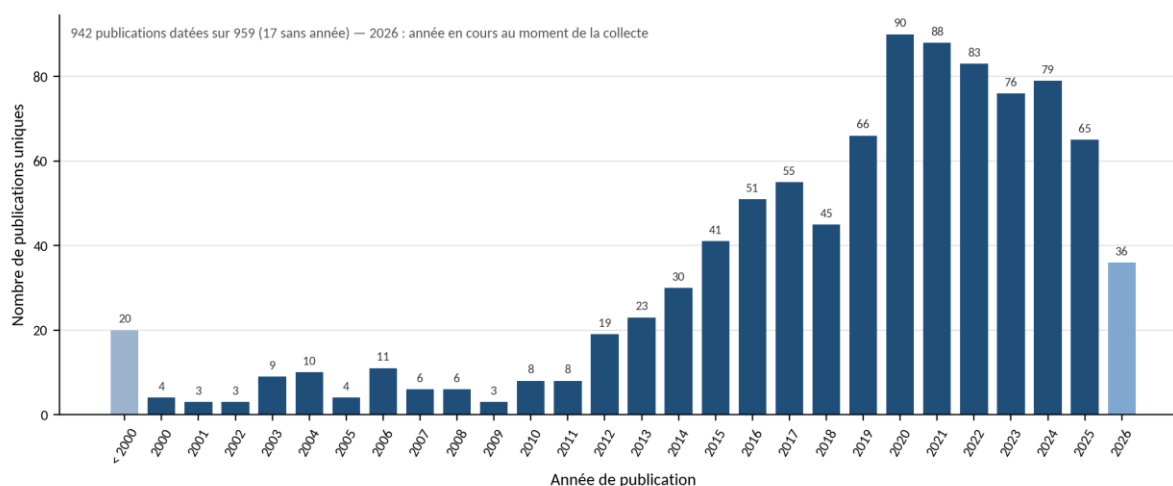


Figure 4 - Répartition des publications uniques par année de publication

La grande majorité des publications datées est postérieure à 2012, avec un maximum autour de 2020–2021. Cette forme reflète à la fois l'activité des chercheurs étudiés et les caractéristiques de la collecte : Google Scholar présente souvent en priorité les publications les plus citées ou les plus récentes, et la collecte limitée de certains profils a pu favoriser une partie de leurs travaux. L'année 2026, en cours au moment de la collecte, est naturellement incomplète. Cette distribution ne doit donc pas être interprétée comme une mesure de l'évolution de la production scientifique de la FSBM.

4. Génération des embeddings

Une fois le corpus consolidé, il s'agit de représenter chaque publication de manière à pouvoir la comparer à une requête par le sens. Ce chapitre présente le principe des embeddings, le modèle retenu, le texte effectivement encodé et le déroulement de la génération des vecteurs.

4.1 Principe des embeddings

Un embedding est une représentation d'un objet, ici un texte, sous la forme d'un vecteur de nombres réels. Les premiers modèles de ce type associaient un vecteur à chaque mot à partir de ses contextes d'apparition [4]. Les modèles récents fondés sur l'architecture Transformer produisent des représentations contextuelles [5], et les modèles de type *sentence embeddings* résument une phrase ou un paragraphe entier en un seul vecteur [6]. L'apprentissage de ces modèles vise une propriété géométrique précise : deux textes de sens proche doivent être représentés par des vecteurs orientés dans des directions voisines.

Cette propriété permet de remplacer la comparaison de mots par une comparaison de vecteurs. Un texte qui parle de « réseaux de neurones pour le diagnostic médical » et un autre qui traite d'« apprentissage profond appliqué à l'imagerie clinique » peuvent n'avoir presque aucun mot en commun, tout en étant représentés par des vecteurs proches.

4.2 Intérêt pour la recherche scientifique

Dans un corpus scientifique pluridisciplinaire, les embeddings présentent plusieurs avantages. Ils rendent la recherche moins dépendante du vocabulaire exact des auteurs, ce qui aide un utilisateur qui ne maîtrise pas la terminologie d'un domaine. Ils permettent de rapprocher des travaux relevant de départements différents lorsqu'ils portent sur des thèmes voisins. Enfin, les vecteurs obtenus constituent une base réutilisable pour d'autres analyses (regroupement thématique, visualisation, recommandation), même si ces usages ne sont pas mis en œuvre dans le présent projet.

4.3 Modèle zembed-1

Le projet utilise exclusivement le modèle `zeroentropy/zembed-1-embedding`, à la révision épinglée `cf13c81f3274394053d166740294f7eea4586f7a`. Aucun modèle de repli n'est prévu : si ce modèle ne peut pas être chargé, le programme s'arrête avec une erreur explicite. `zembed-1` est présenté par ZeroEntropy comme un modèle d'embeddings multilingue à poids ouverts [11], d'environ 4 milliards de paramètres, obtenu par distillation à partir de leur modèle de reranking [12]. Son caractère multilingue est pertinent pour un corpus qui mêle des résumés en anglais et en français.

Le modèle est chargé via la bibliothèque `sentence-transformers` [21], sur CPU et en précision `float32`. Il expose deux fonctions d'encodage distinctes : `encode_document`, pour les textes du corpus, et `encode_query`, pour les requêtes des utilisateurs. Aucune mesure de performance du modèle n'a été réalisée dans le cadre de ce projet ; seules ses propriétés techniques (dimension, validité des vecteurs, cohérence de l'index) ont été vérifiées.

4.4 Préparation du texte

Le texte encodé pour chaque publication est défini par la fonction `document_text` du module `src/embeddings/incremental.py`. Il correspond **uniquement au résumé nettoyé** (`abstract_clean`), sous un format documenté par l'identifiant `abstract_clean/v1`. Le titre, les auteurs, l'année et la revue ne sont pas concaténés au texte encodé : ils sont stockés comme métadonnées associées au vecteur et réapparaissent lors de l'affichage des résultats. Aucune traduction n'est effectuée ; chaque résumé est encodé dans sa langue d'origine.

Ce choix garantit que tous les vecteurs sont calculés sur un type de contenu homogène. Il a toutefois une conséquence : la qualité de la représentation dépend directement de la qualité du résumé collecté, et un résumé tronqué produit une représentation plus pauvre qu'un résumé complet.

4.5 Encodage des documents

Avant tout encodage, une empreinte SHA-256 du texte (préfixée par le format du document) est calculée pour chaque publication. Cette empreinte permet de savoir si un vecteur déjà calculé correspond exactement au texte actuel. Les documents sont ensuite encodés par lots avec `encode_document` (4 documents par lot par défaut). Chaque lot est validé puis sauvegardé immédiatement dans un checkpoint compressé au format NPZ, écrit de façon atomique, ce qui rend la génération interruptible et reprenable.

Le projet a connu deux phases d'indexation. Une première version portait sur les 380 publications de l'échantillon historique. Pour le corpus final, le script `generate_vectors_and_index.py --incremental-final` a comparé les anciens vecteurs au nouveau corpus : un ancien vecteur n'est réutilisé que si l'identifiant de la publication correspond et si le texte encodé est strictement identique. Ce mécanisme a permis de réutiliser **380 vecteurs validés** et de générer **515 nouveaux vecteurs**, soit 895 au total. Un titre identique ne suffit jamais à réutiliser un vecteur.

4.6 Dimension des vecteurs

Chaque vecteur produit par `zembed-1` comporte **2 560 dimensions**. La fonction `validated_vectors` contrôle systématiquement la forme des tableaux, la dimension attendue, l'absence de valeurs non finies (NaN ou infinies) et l'absence de vecteurs nuls. Tout écart provoque l'arrêt du traitement.

4.7 Génération finale

La Figure 5 illustre le processus complet, depuis les 895 publications éligibles jusqu'au stockage des vecteurs dans ChromaDB.



Figure 5 - Processus de génération des embeddings avec zembed-1

Cette figure est cohérente avec l'implémentation : le texte encodé est le résumé nettoyé, l'encodage se fait par lots et chaque publication reçoit un vecteur dense de 2 560 dimensions. Les caractéristiques de l'artefact final, enregistrées dans `data/embeddings/final_manifest.json`, sont reprises dans le Tableau 6.

Tableau 6 - Caractéristiques de l'artefact d'embeddings final

Propriété	Valeur
Modèle	zeroentropy/zembed-1-embedding
Révision	cf13c81f3274394053d166740294f7eea4586f7a
Précision	float32
Texte encodé	abstract_clean/v1 (résumé nettoyé seul)
Dimension	2 560
Nombre de vecteurs	895
Vecteurs réutilisés / générés	380 / 515
Artefact	data/embeddings/final_publication_embeddings.npz
Similarité	cosinus

Le manifeste enregistre aussi une empreinte globale du corpus, calculée à partir des identifiants et des empreintes de contenu. Cette empreinte donne son nom à la collection ChromaDB finale et permet de vérifier que l'index correspond bien au corpus utilisé. En raison de leur taille, le fichier NPZ et l'index ne sont pas versionnés dans le dépôt Git : ils sont distribués séparément dans une archive de la version v1.0.0 du projet.

5. Indexation et recherche sémantique

Disposer de vecteurs ne suffit pas : il faut pouvoir retrouver rapidement ceux qui sont les plus proches d'une requête. Ce chapitre décrit la base vectorielle utilisée, la mesure de similarité et le déroulement d'une recherche.

5.1 Principe de la recherche sémantique

Dans une recherche sémantique, la requête de l'utilisateur est transformée en vecteur par le même modèle que les documents. La recherche revient alors à identifier, dans l'espace vectoriel, les documents dont les vecteurs sont les plus proches de celui de la requête, puis à les classer par proximité décroissante. Cette approche, dite de recherche dense, s'oppose à la recherche lexicale fondée sur la présence des mots de la requête dans le document [3], [7].

5.2 ChromaDB

ChromaDB est une base de données vectorielle open source [16]. Elle stocke, pour chaque élément, un identifiant, un vecteur, le document textuel et un ensemble de métadonnées, et permet d'interroger ces éléments par proximité vectorielle. Le projet l'utilise en mode persistant (`chromadb.PersistentClient`), ce qui signifie que l'index est enregistré sur disque dans le dossier `data/vector_db_final/` et rechargé sans recalcul lors des recherches.

5.3 Index vectoriel

L'index est construit par la fonction `build_index`. Les 895 publications y sont insérées par lots de 32 grâce à l'opération `upsert`, qui évite les doublons en cas de relance. Chaque enregistrement contient l'identifiant de la publication (`article_id`), son vecteur, son résumé nettoyé et des métadonnées : titre, nom et identifiant du chercheur, année, revue, nombre de citations et empreinte de contenu. À la fin de l'insertion, le nombre d'éléments de la collection est comparé au nombre attendu. Le Tableau 7 résume la configuration de l'index.

Tableau 7 - Configuration de l'index vectoriel ChromaDB

Paramètre	Valeur
Mode	Client persistant sur disque
Emplacement	<code>data/vector_db_final/</code>
Collection	<code>fsbm_final_45dc44250169</code>
Structure d'index	HNSW (graphe de voisinage hiérarchique)
Espace de distance	<code>cosinus (hnsw:space = cosine)</code>
Nombre d'éléments	895
Métadonnées par élément	titre, chercheur, année, revue, citations, empreinte

ChromaDB repose sur la structure HNSW (*Hierarchical Navigable Small World*), qui organise les vecteurs en graphes de voisinage hiérarchiques afin d'accélérer la recherche des plus proches

voisins [8]. Il s'agit d'une recherche approximative ; pour un corpus de 895 éléments, la question du passage à l'échelle ne se pose pas réellement, mais l'architecture resterait valable pour un corpus plus important.

5.4 Similarité cosinus

La similarité cosinus mesure l'angle entre deux vecteurs, indépendamment de leur norme. Pour deux vecteurs u et v , elle vaut le produit scalaire de u et v divisé par le produit de leurs normes. Elle est égale à 1 pour deux vecteurs de même direction et diminue à mesure que les directions s'écartent. ChromaDB renvoie une **distance cosinus**, égale à 1 moins la similarité. Le module `semantic_search.py` reconvertit cette distance en score de similarité (`similarity_score = 1 - distance`), arrondi à six décimales, pour que le score affiché soit plus intuitif : plus il est élevé, plus la publication est proche de la requête.

Il faut souligner que ce score est **relatif** : il permet de classer les publications entre elles pour une requête donnée, mais il ne constitue pas une probabilité de pertinence et n'a pas de seuil universel. Un score de 0,5 peut correspondre à un résultat pertinent pour une requête et à un résultat marginal pour une autre.

5.5 Encodage d'une requête

La requête est encodée par la fonction `encode_query`, et non par `encode_document`. Cette distinction est conceptuellement importante. Une requête et un document n'ont pas la même forme : la requête est courte et exprime un besoin (« apprentissage automatique pour le diagnostic médical »), tandis que le document est long et décrit un contenu. Les modèles d'embeddings conçus pour la recherche traitent souvent ces deux types d'entrée de façon asymétrique, afin de rapprocher une question des textes qui y répondent plutôt que des textes qui lui ressemblent formellement. Utiliser les deux fonctions prévues par le modèle respecte cette conception. Le code vérifie en outre que le vecteur de la requête a bien la même dimension que ceux de l'index.

5.6 Recherche Top-K

Le déroulement d'une recherche est illustré par la Figure 6.

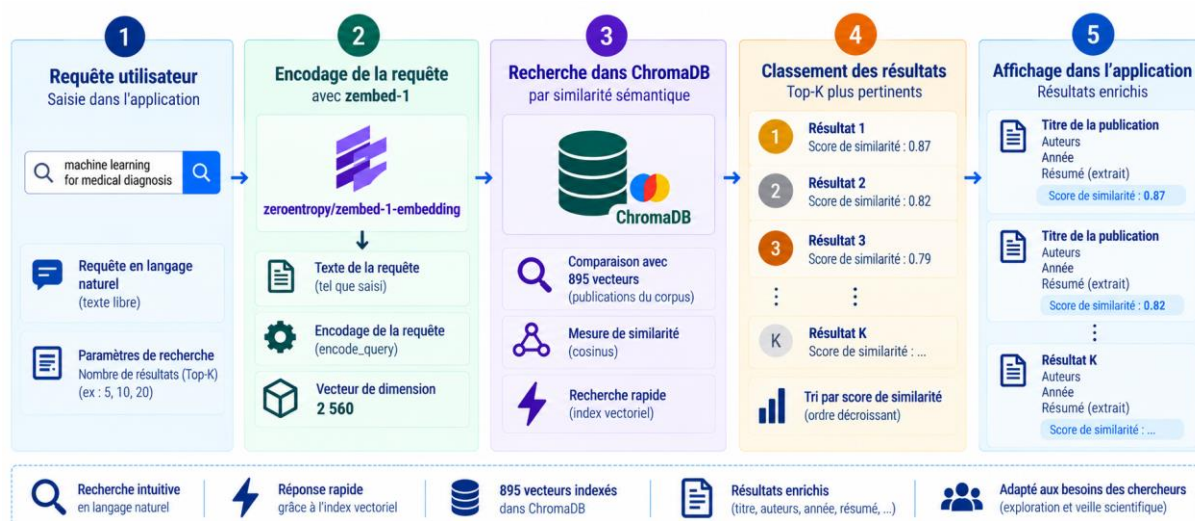


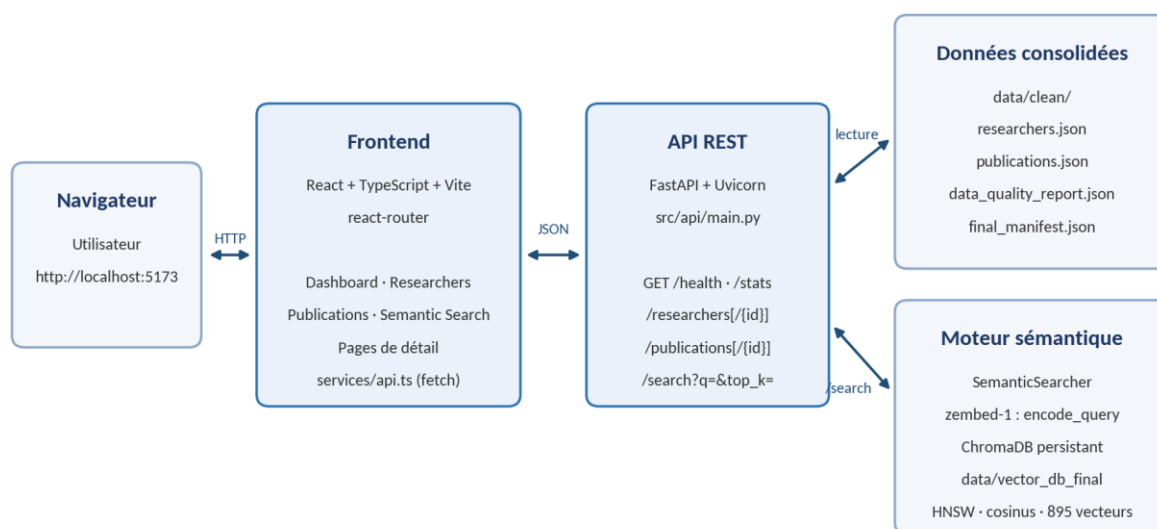
Figure 6 - Processus de recherche sémantique et classement des publications par similarité

La recherche suit les étapes suivantes : la requête saisie est validée (texte non vide, nombre de résultats K positif) ; elle est encodée par `encode_query` en un vecteur de 2 560 dimensions ; ChromaDB recherche les K éléments les plus proches selon la distance cosinus ; les distances sont converties en scores de similarité ; les résultats sont triés par score décroissant et numérotés ; enfin, les métadonnées de chaque publication sont récupérées pour l'affichage. L'API complète ces résultats avec les informations du dataset consolidé (auteurs, date, résumé original).

Les scores visibles sur la figure (0,87, 0,82, 0,79) sont purement illustratifs et ne proviennent pas d'une exécution réelle. Les résultats effectivement obtenus avec l'application sont présentés au chapitre 6 à partir de captures d'écran réelles.

6. Conception et réalisation de l'application

Pour rendre le corpus et la recherche sémantique accessibles sans passer par la ligne de commande, le projet comprend une application web de démonstration composée d'une API REST et d'une interface utilisateur. Ce chapitre présente leur architecture et leurs fonctionnalités, illustrées par des captures d'écran de l'application en fonctionnement. La Figure 7 présente l'architecture logicielle réelle de l'application.



CORS limité aux origines locales (ports 3000 et 5173) · méthodes GET uniquement · modèle chargé à la première recherche

Figure 7 — Architecture logicielle de l'application de démonstration

L'architecture comporte trois niveaux. Le navigateur affiche l'interface React, qui interroge l'API FastAPI par des requêtes HTTP renvoyant du JSON. L'API lit les fichiers consolidés du dossier `data/clean/` et le manifeste final ; pour la recherche uniquement, elle fait appel au moteur sémantique, qui charge le modèle zembed-1 et interroge l'index ChromaDB. Il n'y a pas de base de données relationnelle intermédiaire : les fichiers JSON sont chargés en mémoire une seule fois grâce à un cache (`lru_cache`).

6.1 Backend FastAPI

Le backend est implémenté dans `src/api/main.py` avec FastAPI [17] et servi par Uvicorn. Les schémas de réponse sont décrits par des modèles Pydantic, ce qui fournit une validation automatique et une documentation interactive accessible à l'adresse `/docs`. Le Tableau 8 liste les endpoints réellement exposés.

Tableau 8 - Endpoints de l'API REST

Endpoint	Description
GET /health	État du service, nom du projet et modèle utilisé
GET /stats	Compteurs du corpus et de l'index, lus dans le rapport de qualité et le manifeste

Endpoint	Description
GET /researchers	Liste paginée des chercheurs (page, page_size ≤ 100)
GET /researchers/{researcher_id}	Profil d'un chercheur et publications associées
GET /publications	Liste paginée des publications, filtrable par year et researcher_id
GET /publications/{publication_id}	Détail d'une publication
GET /search?q=...&top_k=5	Recherche sémantique (top_k entre 1 et 20)

Plusieurs choix de conception méritent d'être relevés. L'API est **en lecture seule** : seule la méthode GET est autorisée, et la politique CORS est limitée aux origines locales du serveur de développement (ports 3000 et 5173). Le modèle d'embeddings n'est **chargé qu'à la première recherche**, de manière paresseuse et protégée par un verrou ; les autres endpoints répondent donc sans charger le modèle. Avant ce chargement, l'API vérifie que le manifeste final correspond exactement au modèle, à la révision, au format de document et à l'index attendus. Enfin, les erreurs sont traduites en codes HTTP explicites : 404 pour une ressource absente, 422 pour une requête vide, 503 lorsqu'un artefact ou le service de recherche est indisponible.

6.2 Frontend

L'interface est développée avec React [18] et TypeScript [19], et construite avec l'outil Vite [20]. La navigation entre les pages est assurée par react-router. Le code est organisé en pages (pages/), composants réutilisables (components/, par exemple les cartes de chercheur, de publication et de résultat), un hook générique de chargement de données (hooks/useResource.ts), les types TypeScript des réponses de l'API (types/api.ts) et un module de service (services/api.ts) qui centralise tous les appels HTTP. L'adresse de l'API est configurable par la variable VITE_API_BASE_URL. Le Tableau 9 présente les routes de l'application.

Tableau 9 - Pages de l'interface web

Route	Page	Endpoint(s) utilisé(s)
/	Dashboard	/stats
/researchers	Researchers	/researchers
/researchers/:id	Détail chercheur	/researchers/{id}
/publications	Publications	/publications
/publications/:id	Détail publication	/publications/{id}
/search	Semantic Search	/search

Chaque page gère explicitement trois états : chargement, erreur et absence de résultat. Les requêtes en cours sont annulées lorsque l'utilisateur change de page, grâce à `AbortController`, ce qui évite l'affichage de réponses obsolètes. En cas d'échec de l'API, l'interface affiche un message d'erreur plutôt que des données fictives.

6.3 Tableau de bord

La page d'accueil, présentée dans la Figure 8, introduit le projet et affiche les principaux indicateurs du corpus.

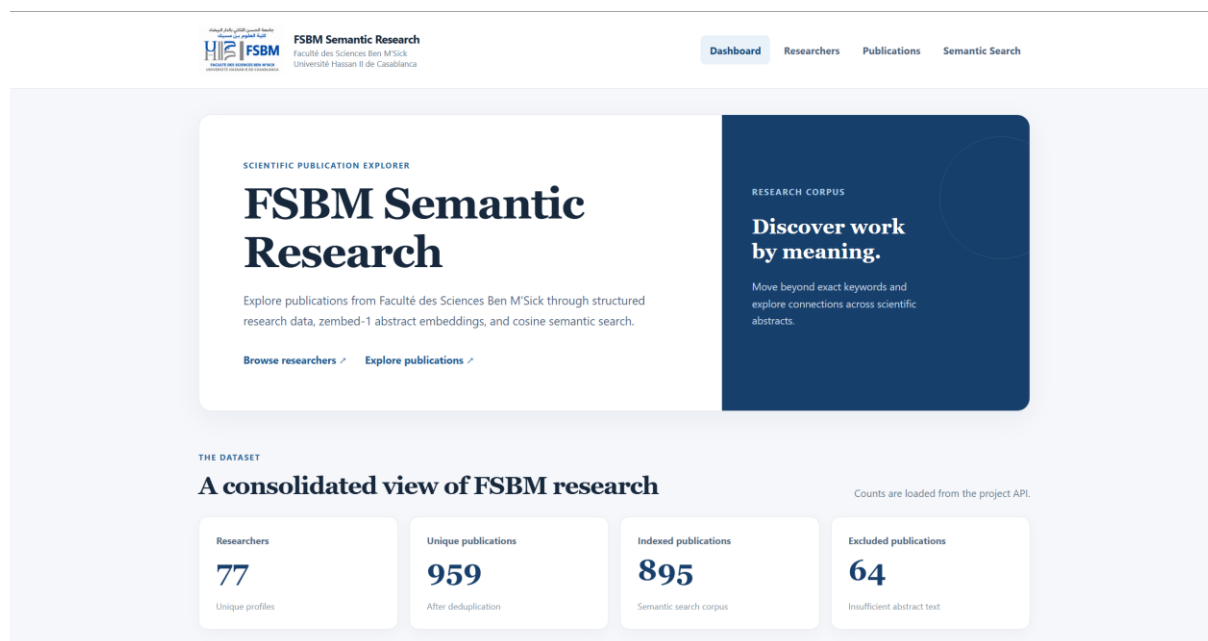


Figure 8 - Tableau de bord de l'application FSBM Semantic Research

Les quatre compteurs affichés (77 chercheurs, 959 publications uniques, 895 publications indexées et 64 publications exclues) ne sont pas codés en dur dans l'interface : ils sont chargés depuis l'endpoint `/stats`, lui-même alimenté par le rapport de qualité et le manifeste des embeddings. Le tableau de bord reflète donc toujours l'état réel des artefacts. Des liens permettent d'accéder directement aux listes de chercheurs et de publications.

6.4 Consultation des chercheurs

La page Researchers, illustrée par la Figure 9, liste les profils du dataset consolidé.

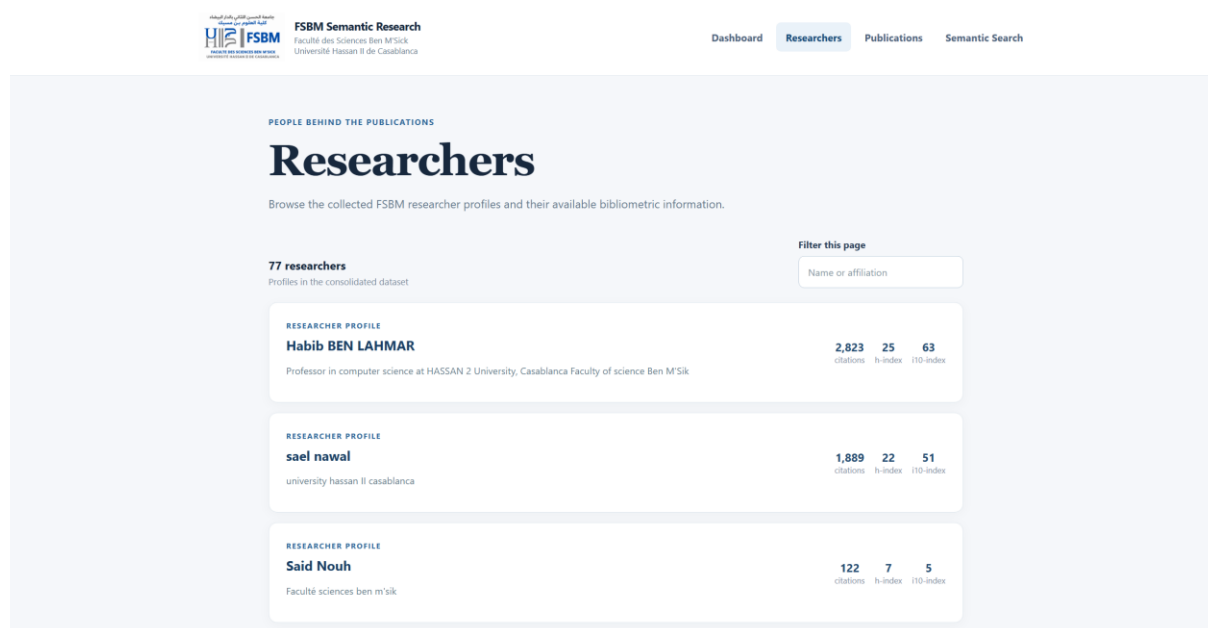


Figure 9 - Consultation des chercheurs et de leurs indicateurs scientifiques

Chaque carte affiche le nom du chercheur, son affiliation telle qu'elle est déclarée sur Google Scholar, ainsi que ses citations, son indice h et son indice i10. La capture montre par exemple un profil totalisant 2 823 citations avec un indice h de 25. Les noms et affiliations sont affichés tels qu'ils ont été collectés, sans correction de casse ni harmonisation, ce qui explique certaines différences de présentation. Un champ de filtrage permet de restreindre les profils affichés sur la page courante par nom ou par affiliation ; un clic sur une carte ouvre la page de détail du chercheur avec ses publications associées. Dans le corpus, le nombre de citations par profil va de 0 à 6 932, pour une médiane de 447, et l'indice h de 0 à 43, pour une médiane de 11.

6.5 Consultation des publications

La page Publications, présentée dans la Figure 10, donne accès aux 959 publications uniques.

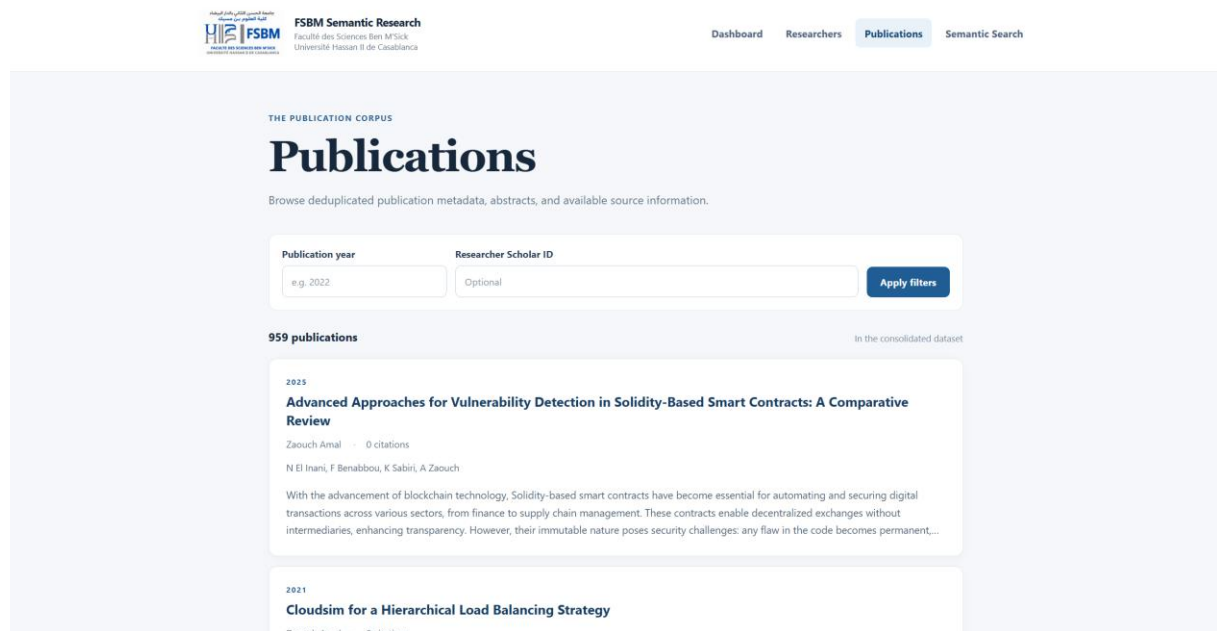


Figure 10 - Consultation et exploration des publications scientifiques

Chaque publication est présentée avec son année, son titre, le chercheur de rattachement, son nombre de citations, la liste des auteurs et un extrait du résumé. Un formulaire permet de filtrer les publications par année (valeur à quatre chiffres, validée côté client) et par Scholar ID ; le filtrage est exécuté par l'API. Les publications sans résumé, exclues des embeddings, restent visibles sur cette page, conformément au principe selon lequel l'exclusion ne concerne que la vectorisation.

6.6 Recherche sémantique

La page Semantic Search constitue le cœur fonctionnel de l'application. La Figure 11 montre une recherche réelle effectuée avec la requête « Natural language processing ».

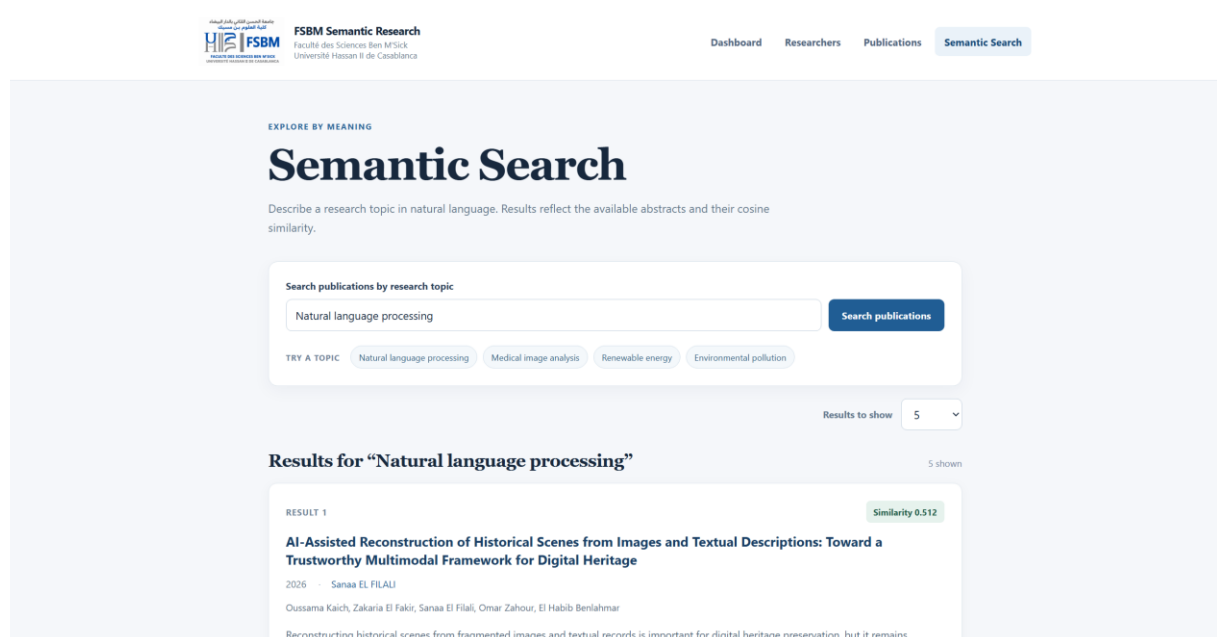


Figure 11 - Exemple de recherche sémantique et classement des publications selon leur similarité

L'utilisateur saisit un sujet en langage naturel ou choisit l'un des thèmes proposés, puis sélectionne le nombre de résultats à afficher (5, 10 ou 20). La requête n'est envoyée qu'après validation par l'utilisateur. Chaque résultat affiche son rang, son score de similarité, son titre, son année, le chercheur de rattachement, les auteurs et le début du résumé.

Dans l'exemple présenté, cinq résultats sont affichés. Le premier est une publication de 2026 rattachée au profil de Sanaa El Filali, intitulée *AI-Assisted Reconstruction of Historical Scenes from Images and Textual Descriptions*, avec une similarité de 0,512. Ce résultat est cohérent sans être parfait : l'article traite de descriptions textuelles dans un cadre multimodal, donc d'un sujet voisin du traitement du langage naturel, mais il ne porte pas exclusivement sur le NLP. Le score, modéré, rappelle que la similarité mesure une proximité relative au sein du corpus disponible. Une note affichée sous les résultats précise d'ailleurs que le classement ne garantit pas la pertinence de chaque résultat.

7. Tests, validation et résultats

Ce chapitre présente les tests automatisés, les validations propres aux embeddings et à l'index, puis une analyse qualitative des résultats de la recherche sémantique.

7.1 Tests automatisés

Les tests Python sont écrits avec le module standard `unittest` et regroupés dans le dossier `tests/`. Ils sont conçus pour s'exécuter **hors ligne** et **sans charger le modèle de 4 milliards de paramètres** : les accès à Google Scholar, à OpenAlex et au modèle sont simulés par des objets factices. Les tests frontend utilisent Vitest [22] et Testing Library, avec des réponses d'API simulées. Le Tableau 10 présente les résultats obtenus lors de l'exécution des suites de tests dans l'environnement de développement du projet.

Tableau 10 - Résultats des validations automatisées

Type de validation	Commande	Résultat
Tests Python (46 tests)	<code>python -m unittest discover -s tests</code>	46 réussis
Tests frontend (5 tests)	<code>npm test</code>	5 réussis
Vérification des types TypeScript	<code>tsc --noEmit</code>	Validation réussie
Build de production Vite	<code>vite build</code>	Réussi

Les 46 tests Python couvrent l'ensemble des modules du pipeline. Leur répartition, présentée dans le Tableau 11, montre que les parties les plus sensibles, à savoir l'enrichissement PDF et la gestion incrémentale des embeddings, sont aussi les plus testées.

Tableau 11 - Répartition des tests Python par module

Fichier de test	Tests	Aspects vérifiés
<code>test_scraper.py</code>	1	Reprise après checkpoint sans recollecter les profils terminés
<code>test_batch2_validation.py</code>	1	Recouvrement d'identifiants et similarité des noms
<code>test_batch3_selection.py</code>	3	Sélection du lot 3, déduplication des identifiants, checkpoint séparé
<code>test_cleaner.py</code>	4	Normalisation Unicode/HTML, règles de doublons, exclusions, sorties JSON/Parquet
<code>test_merge_raw_data.py</code>	3	Fusion des schémas, aller-retour Parquet, profils partiels
<code>test_incremental_embeddings.py</code>	8	Réutilisation des vecteurs, reprise, rejet des dimensions ou modèles invalides
<code>test_semantic_search.py</code>	5	Validation des requêtes, dimensions, tri des résultats, index cosinus
<code>test_api.py</code>	5	Endpoints, erreurs 404/422, recherche simulée, chargement différé du modèle

Fichier de test	Tests	Aspects vérifiés
test_pdf_enrichment.py	16	URL, robots.txt, validation PDF, reprise, limitation de débit
Total	46	

Les cinq tests frontend vérifient notamment l’affichage des statistiques réelles, l’absence de recherche avant soumission, l’affichage d’une erreur de l’API sans résultats fabriqués et la construction des URL d’appel.

7.2 Validation des embeddings et de l’index

Au-delà des tests unitaires, la cohérence des artefacts finaux est contrôlée à plusieurs niveaux, dans le code de génération et dans le notebook de démonstration :

- 895 vecteurs, soit exactement le nombre de publications éligibles, tous de dimension 2 560 ;
- aucun vecteur ne contient de valeur non finie, et aucun n’est nul ;
- chaque vecteur est lié à un identifiant de publication unique et à l’empreinte de son texte ;
- les identifiants de l’index correspondent aux 895 lignes éligibles du fichier Parquet ;
- la collection ChromaDB contient 895 éléments, nombre comparé au manifeste à chaque chargement du moteur de recherche.

Ces vérifications sont bloquantes, et le notebook de démonstration les reprend en lecture seule, sans recharger le modèle.

7.3 Analyse des résultats de la recherche sémantique

L’évaluation de la recherche sémantique reste qualitative. Aucun jeu de requêtes annotées avec des jugements de pertinence n’a été constitué ; il n’est donc pas possible de calculer des métriques standard de recherche d’information comme la précision ou le NDCG. Le script `src/search/evaluate_queries.py` définit cinq requêtes de référence couvrant des domaines variés (diagnostic médical, NLP, énergies renouvelables, géologie environnementale, prédiction du cancer), mais ses scores ne sont pas archivés dans le dépôt et ne sont donc pas reproduits ici.

Les observations disponibles conduisent à un constat nuancé. La recherche fonctionne de bout en bout et renvoie des publications thématiquement proches de la requête (Figure 11). Cependant, **toutes les requêtes ne donnent pas des résultats également pertinents**. Lors de la validation du projet, la requête « cancer prediction using machine learning » a principalement renvoyé des articles généraux sur l’apprentissage automatique plutôt que des travaux spécifiques au cancer. Ce comportement ne traduit pas nécessairement un défaut du modèle : il s’explique plausiblement par un corpus qui contient peu de publications sur ce sujet précis. La recherche sémantique ne peut renvoyer que ce qui existe dans l’index ; face à une requête peu couverte, elle renvoie les documents les moins éloignés, qui peuvent n’être que partiellement liés.

La qualité des résumés (dont plus de la moitié sont tronqués), la formulation de la requête et l'absence du titre dans le texte encodé influencent aussi la pertinence, ce qui justifie une interprétation prudente des scores.

8. Enrichissement PDF et limitations

Ce dernier chapitre présente un module complémentaire, l'enrichissement par des PDF en accès ouvert, puis rassemble les limitations du projet. Il est important de distinguer clairement le **dataset principal**, sur lequel reposent les embeddings, l'index et l'application, et l'**enrichissement PDF**, qui constitue une étape optionnelle et indépendante.

8.1 Module d'enrichissement PDF optionnel

Le module `src/enrichment/pdf_enrichment.py` cherche à associer aux publications des fichiers PDF librement accessibles. Il exploite d'abord les métadonnées existantes, notamment les URL contenant un DOI, puis interroge les métadonnées d'accès ouvert d'OpenAlex [10], par DOI ou par correspondance stricte du titre, de l'année et des auteurs disponibles. Par défaut, le module se limite à la **découverte** d'URL candidates ; le téléchargement n'a lieu que si l'option `--download` est explicitement fournie.

Les règles appliquées sont strictes. Une page d'article ordinaire n'est pas considérée comme un PDF. Le fichier `robots.txt` des sites est respecté, les redirections ne sont pas suivies automatiquement, les réponses de type HTML ou texte sont rejetées, et un fichier n'est validé que s'il commence par la signature `%PDF-` et a une taille acceptable. Aucune restriction d'accès, aucun paywall et aucun CAPTCHA n'est contourné. Surtout, **cette étape ne modifie ni le dataset nettoyé, ni les embeddings, ni l'index, ni l'application** : ses résultats sont écrits dans le dossier séparé `data/enriched/`.

8.2 Résultats de l'enrichissement

Le rapport `pdf_enrichment_report.json` documente une exécution sur l'ensemble du corpus. Les 959 publications ont été sélectionnées, mais la limitation de débit imposée par la source a interrompu l'exécution après **231 publications effectivement vérifiées**. Les 728 autres n'ont pas été examinées lors de cette exécution. Le Tableau 12 détaille les statuts obtenus.

Tableau 12 - Statuts de l'exécution d'enrichissement PDF (231 publications vérifiées)

Statut	Publications
PDF téléchargé et validé	11
Aucun PDF trouvé	156
Accès interdit par robots.txt	38
Réponse invalide (fichier non PDF)	17
Accès restreint	6
Erreur	2
Limitation de débit	1
Total vérifié	231

Sur les 231 publications vérifiées, 74 URL candidates de PDF ont été identifiées, mais seuls **11 PDF ont été téléchargés et validés**. La différence s'explique par les réponses invalides, les interdictions robots.txt et les restrictions d'accès : une URL candidate est une piste, non un document obtenu. Ces résultats confirment que l'accès au texte intégral reste marginal dans ce contexte. Ils expliquent aussi pourquoi le dataset principal indique zéro URL de PDF validée : il s'agit de deux étapes distinctes, qui mesurent des choses différentes.

8.3 Limitations générales

Les limitations du projet ont été mentionnées au fil des chapitres. Le Tableau 13 les rassemble, en indiquant pour chacune la mesure prise dans le projet.

Tableau 13 - Limitations rencontrées et mesures mises en place

Limitation	Effet	Mesure prise
Dépendance à Google Scholar	Pas d'API officielle, structure imposée par la source	Bibliothèque scholarly, provenance conservée
Rate limiting et CAPTCHA	Collecte interrompue, profils partiels	Délais, backoff, arrêt au blocage, checkpoints et reprise
Couverture partielle	77 profils sur 93 ; 31 profils limités	Statut de collecte enregistré par profil
Métadonnées incomplètes	Années, revues, URL manquantes	Valeurs laissées nulles, jamais estimées
Résumés absents ou tronqués	64 publications non vectorisées ; 490 résumés tronqués	Exclusion documentée, texte original conservé
Affiliation non vérifiée	Appartenance à la FSBM incertaine	Audit de couverture, mise en garde explicite
Taille du corpus	Certains sujets peu ou pas couverts	Présentation comme échantillon partiel
Pertinence variable	Résultats généraux pour certaines requêtes	Scores présentés comme relatifs
Coût du modèle	Modèle d'environ 4 milliards de paramètres	Chargement différé, vecteurs réutilisés, tests sans modèle

Plusieurs de ces limitations méritent un commentaire. La **dépendance à Google Scholar** est la plus structurante : elle détermine le volume, la qualité et la complétude des données, et le respect de ses restrictions a volontairement été privilégié par rapport au volume. La **disponibilité variable des résumés** a un effet direct sur la recherche, puisque seul le résumé est encodé : une publication sans résumé est invisible pour la recherche sémantique, et un résumé tronqué produit une représentation incomplète. La **difficulté de vérifier automatiquement l'affiliation institutionnelle** interdit de présenter le corpus comme la production de la FSBM : toute conclusion institutionnelle exigerait une vérification auprès de sources officielles.

Enfin, le **coût computationnel du modèle** est réel : la première recherche après le démarrage de l'API peut être lente, et malgré les mesures prises (chargement unique, réutilisation des vecteurs), ce coût reste un frein à un déploiement sur une infrastructure modeste.

Conclusion générale et perspectives

Ce projet partait d'une question simple en apparence : comment explorer par le sens les publications associées aux chercheurs de la FSBM ? Y répondre a nécessité de constituer un corpus à partir d'une source publique contrainte, de le mettre en qualité, de le représenter sémantiquement et de le rendre accessible.

La solution réalisée prend la forme d'un pipeline complet et reproductible. La collecte avec `scholarly` a permis de rassembler 77 profils et 1 044 enregistrements de publications, avec des mécanismes de robustesse (délais, backoff, checkpoints atomiques, reprise) et sans aucun contournement des restrictions de Google Scholar. Le prétraitement a produit un corpus de 959 publications uniques après fusion de 85 doublons, selon des règles de déduplication prudentes, et a identifié 895 publications disposant d'un résumé exploitable. Ces 895 publications ont été encodées avec le modèle `zeroentropy/zembed-1-embedding` en vecteurs de 2 560 dimensions, puis indexées dans ChromaDB avec une distance cosinus. Une API FastAPI et une interface React rendent l'ensemble consultable, et 51 tests automatisés vérifient les différents modules.

L'intérêt de la recherche sémantique apparaît clairement dans un corpus pluridisciplinaire : elle permet de retrouver des travaux proches sans connaître leur vocabulaire exact. Le projet montre aussi que la valeur d'un tel outil dépend autant de la rigueur du travail sur les données (traçabilité, conservation des valeurs originales, transparence sur les exclusions) que du modèle utilisé.

Les limites du travail ont été exposées sans les minimiser : collecte partielle, métadonnées incomplètes, résumés absents ou tronqués, appartenance institutionnelle non vérifiée, pertinence variable selon les requêtes et coût du modèle. Elles ouvrent plusieurs perspectives d'amélioration, qui constituent des pistes de travail futures et non des fonctionnalités actuelles :

- **élargir le corpus**, en complétant les profils limités et en intégrant les identifiants non encore collectés ;
- **mobiliser d'autres sources bibliographiques**, par exemple OpenAlex, déjà utilisé pour l'enrichissement, afin de compléter les métadonnées et de réduire la dépendance à Google Scholar ;
- **poursuivre l'enrichissement en accès ouvert** sur les 728 publications non encore vérifiées, pour disposer de résumés complets ou de textes intégraux ;
- **améliorer la qualité des métadonnées**, notamment la vérification des affiliations à partir de sources institutionnelles ;
- **ajouter des filtres à la recherche sémantique** (année, chercheur, domaine) et étudier l'intérêt d'intégrer le titre au texte encodé ;
- **développer des analyses bibliométriques et des visualisations** des relations entre chercheurs et domaines, par exemple par regroupement des vecteurs ;
- **déployer l'application** sur une infrastructure accessible aux membres de la faculté ;
- **évaluer systématiquement la pertinence** à l'aide d'un jeu de requêtes annotées, afin de mesurer objectivement la qualité de la recherche.

Au-delà de ses résultats chiffrés, ce projet a permis de mettre en pratique une chaîne complète de traitement de données textuelles, de la collecte à l'application, en tenant compte des contraintes réelles de qualité et d'accès aux données.

Références bibliographiques

- [1] L. Bornmann and R. Mutz, "Growth rates of modern science: A bibliometric analysis based on the number of publications and cited references," *Journal of the Association for Information Science and Technology*, vol. 66, no. 11, pp. 2215–2222, 2015.
- [2] G. Salton, A. Wong and C. S. Yang, "A vector space model for automatic indexing," *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975.
- [3] C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [4] T. Mikolov, K. Chen, G. Corrado and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781, 2013.
- [5] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [7] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [9] J. E. Hirsch, "An index to quantify an individual's scientific research output," *Proceedings of the National Academy of Sciences*, vol. 102, no. 46, pp. 16569–16572, 2005.
- [10] J. Priem, H. Piwowar and R. Orr, "OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts," arXiv:2205.01833, 2022.
- [11] ZeroEntropy, "Models — zembed-1," documentation en ligne. [En ligne]. Disponible : <https://docs.zeroentropy.dev/models>
- [12] ZeroEntropy, "Introducing zembed-1: The world's best text-embedding model," 2 mars 2026. [En ligne]. Disponible : <https://zeroentropy.dev/articles/introducing-zembed-1-the-worlds-best-multilingual-text-embedding-model>
- [13] scholarly contributors, "scholarly: Retrieve author and publication information from Google Scholar," dépôt logiciel. [En ligne]. Disponible : <https://github.com/scholarly-python-package/scholarly>
- [14] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference (SciPy)*, 2010, pp. 56–61.
- [15] The Apache Software Foundation, "Apache Parquet documentation." [En ligne]. Disponible : <https://parquet.apache.org/docs/>
- [16] Chroma, "Chroma documentation." [En ligne]. Disponible : <https://docs.trychroma.com>
- [17] S. Ramírez, "FastAPI documentation." [En ligne]. Disponible : <https://fastapi.tiangolo.com>
- [18] Meta Platforms, "React documentation." [En ligne]. Disponible : <https://react.dev>
- [19] Microsoft, "TypeScript documentation." [En ligne]. Disponible : <https://www.typescriptlang.org/docs/>
- [20] Vite, "Vite guide." [En ligne]. Disponible : <https://vite.dev/guide/>

- [21] Sentence Transformers, "Sentence Transformers documentation." [En ligne]. Disponible : <https://sbert.net>
- [22] Vitest, "Vitest documentation." [En ligne]. Disponible : <https://vitest.dev>